

Aura — Developer Onboarding

A technical overview of the systems, how they are wired, and what they look like from the player's side. Written for someone about to start working on the repo.

Repo `github.com/RoteRiesenRobbe/aura` (workspace dir `aurahunter`) **State** 2026-07-22, branch `main`

Live `aura-game.duckdns.org`

1. What the project is

Aura is a browser-based top-down multiplayer MMO, built on a fork of the *Berryhunter* survival game. The survival systems (hunger, temperature, crafting, inventory) were stripped out; what remains of the inheritance is the load-bearing infrastructure: the Go game server, the WebSocket + FlatBuffers netcode, the 2D physics, and the PixiJS renderer. Everything above that is new.

The design premise, and the thing that shapes most of the code: **players and NPCs interact exclusively through auras** — circular effect fields that apply automatically to whatever is inside them. There is no target selection and no attack button for the main damage loop. Positioning and cooldown timing are the only skill expressions. Mobs run the exact same skill system as players.

Every entity — player, mob, NPC — has exactly **one resource** (integer HP). It is health, mana, and cost pool at once. Zero means death.

~26.7k

GO LOC (PROD)

~21.9k

GO LOC (TESTS)

~18.9k

TYPESCRIPT LOC

50

GO PACKAGES

83

SKILL JSONS

50

MOB JSONS

30 Hz

SERVER TICK

2. Repo layout, build, run

PATH	WHAT LIVES THERE
<code>backend/</code>	Go game server. Binary is <code>aurad</code> . Entry point <code>cmd/aurad/</code> ; all game code under <code>pkg/aura/</code> .
<code>frontend/</code>	TypeScript + webpack + PixiJS browser client. Feature-module layout under <code>src/features/</code> .
<code>api/</code>	The shared contract <i>and</i> all game content: FlatBuffers schemas (<code>api/schema/</code>) plus JSON definitions for skills, mobs, items, factions, recipes, props, milestones, zones.
<code>docs/</code>	GDD, TDD, architecture, per-feature plan docs, content catalogues, authoring manuals. Heavily used — most design decisions are recorded there with rationale.
<code>devops/</code>	Deploy script, systemd unit, load-test harness notes, server runbook.
<code>CLAUDE.md</code>	The live status document. What was last finished, what is next, every open placeholder value. Read this first each session.

GETTING IT RUNNING

```
cp backend/conf.local-windows.json backend/conf.json # or conf.default.json
make -C backend build                               # → backend/aurad
cd backend && ./aurad -dev                            # port 2000

cd frontend && npm install && npm run start            # webpack HMR, port 2001
# open:
```

```
http://localhost:2001/?token=plz&wsUrl=ws://localhost:2000/game
# optional: &develop (dev panel), &start-cmds=GOD,XP 5000,...
```

`backend/tokens.list` must exist with at least one token for in-game commands (cheats) to work. Dev cheats available in-game: `GOD`, `WARP x y`, `SPEED`, `XP`, `SKILL`, `ANNOUNCE`, `THREAT`.

THREE BUILD GOTCHAS THAT WILL COST YOU AN HOUR EACH

- `go build ./...` type-checks but does **not** refresh `./aurad`. A running `-dev` server keeps executing the old binary. Use `make -C backend build`.
- Content JSON is `go:embed`ed via `backend/pkg/api/`, refreshed by `make cp-defs`. For content iteration, boot `./aurad -dev -content ../api` to read the repo tree directly — then a restart is enough, no rebuild. The boot log prints which source it used.
- Webpack HMR only watches `frontend/src`. Regenerated FlatBuffers bindings (`api/schema/js/`) and webpack *config* changes need a dev-server restart.

3. Backend architecture

3.1 The loop

The server is an **ECS** (github.com/EngoEngine/ecs) running a **fixed 30 Hz tick, single-threaded**.

`core/game.go:Loop()` is literally `for { g.update(); <-ticker.C }` — every registered system's `Update()` runs sequentially in one goroutine, with a 33 ms budget per tick. Exceeding it logs `Overload! Systems at: N%`.

Single-threadedness is the structural ceiling. One core caps total simulation work regardless of how tight each piece is. The designed escape hatch is one physics `Space` per zone (each on its own goroutine or process), not micro-optimisation. Today the whole world is one `Space`.

3.2 The systems

Registered in order in `core.NewGameWith()`. Execution order is the engine's priority sort — **do not re-sort** `World.Systems()`, it returns the live slice (there is a regression test pinning this).

SYSTEM	RESPONSIBILITY
PhysicsSystem	Owns the <code>phy.Space</code> : movement integration, broadphase, collision sets.
NetSystem	Per-player area-of-interest snapshot assembly + FlatBuffers marshalling + websocket send.
InputSystem	Drains client input into player intent (movement vector, aura slot, cooldown activations).
MobSystem	Spawning, respawn timers, AI state machine (idle/wander/chase/attack/leash), obstacle steering.
NpcSystem	Peaceful NPCs: proximity trigger, chat bubbles, skill teaching.
ChatSystem	Chat + server-wide announcements (also the <code>Announcer</code> for encounters and commands).
EncounterSystem	Scripted boss encounters (phases, threat table, adds).
Pre/PostUpdateSystem	Per-tick state reset and commit brackets around the gameplay systems.
UpdateSystem	Player entity update: regen, death checks, level-ups.
SkillSystem	The core. Resolves every equipped skill of every entity per tick. ~1,600 lines, the deliberate complexity sink.
StatusEffectsSystem	Transient visual/status flags (damaged, burst-fired, regenerating) fed to the wire.
ConnectionStateSystem	Join/leave, death state, respawn, and the reconnect-token stash (reload restores your character for ~10 min).
CommandSystem	Token-gated cheat/admin commands.
EquipSystem	Aura slot loadout changes, skill point spend/respec.
DecaySystem	Corpse and temporary-entity lifetimes.

3.3 Physics and the two scaling properties

`pkg/aura/phy/` is a small purpose-built 2D layer: circles and AABBs, a uniform spatial hash with 10-unit cells, layer masks. Static shapes (props, walls, world boundary) live in a persistent static grid inserted once; dynamic shapes go into a grid that is cleared and reused each tick. Collision inside a cell is brute force, $O(k^2)$ over the k shapes in it.

Two consequences you should internalise before touching anything performance-adjacent:

- **Large shapes are the cost amplifier.** Roughly three shapes per entity (mob: body + aura + aggro sensor; player: body + aura + viewport). A player's viewport is a 20×12 box spanning 4–6 cells, and it is brute-forced in *every* cell it overlaps.
- **Density, not headcount, is the limit.** Cost is $\sum k_c^2$ per cell, so N players spread over a zone is cheap and N players piled on one screen is quadratic.

The network layer already avoids the classic $O(\text{players} \times \text{entities})$ MMO trap: each player is sent only the entities intersecting their viewport shape. That is the single most important scaling property in the codebase — and the same reason clustering hurts twice (broadphase *and* send-set both explode).

MEASURED, NOT THEORETICAL

There is a load-test harness (`cmd/loadbot`, see `devops/loadtest.md`) that drives real websocket clients with real skills. Current numbers: spread players are fine well past 120; **clustered players hit a wall around 60-70**, and the wall is the single-threaded per-player snapshot encoding. Mobs cost more than auras. Re-measure before quoting any of these — they predate the most recent allocation work.

4. The skill / aura system — the part that matters

Everything gameplay-facing is expressed as a **skill**. Skills are JSON, live in `api/skills/` (47 player + 36 mob definitions), and are loaded into a registry at boot with **hard-fail validation**. Players and mobs both carry a `SkillComponent`; the mechanics are identical for both.

4.1 Three categories

CATEGORY	SEMANTICS
<code>active_aura</code>	Toggleable field with a visible in-world radius. Exactly one is active at a time . The slots are a loadout — several equipped, one on, switchable mid-fight. Build variety comes from loadout + switch timing, not from stacking auras.
<code>passive</code>	Always on, these <i>do</i> run in parallel. Stat multipliers, resistances, flat bonuses.
<code>cooldown</code>	Manually fired ability with a cooldown timer. Self-healing is a cooldown, never an aura.

4.2 Anatomy of a definition

A skill is `id` + `name` + `category` + `maxLevel` + a list of **effects**. There are **22 effect types**:

`damage_aura` · `heal_aura` · `hot_aura` · `dot_aura` · `shield_aura` · `resist_aura` · `slow_aura` · `light_aura` · `stat_multiplier` · `resist_passive` · `instant_damage` · `instant_dot` · `instant_hot` · `instant_shield` · `self_heal` · `spawn` · `taunt` · `detaunt` · `dash` · `recall` · `revive` · `tick_rate`

```
{
  "id": 1, "name": "Damage", "category": "active_aura", "maxLevel": 5,
  "effects": [{
    "type": "damage_aura",
    "radius": 1.0, "radiusPerLevel": 0.0,
    "damageHP": 14, "damageHPPerLevel": 3.2, "variance": 0.15,
    "targetsEnemies": true, "targetsAllies": false,
    "selector": "nearest", "maxTargets": 1,
    "tickInterval": 40
  }]
}
```

Effect semantics stay in Go — **this was an explicit decision**: no scripting engine for effects. New behaviour means a new effect type, not a new expression language. A constrained expression layer is parked behind an explicit trigger that has not fired.

4.3 Targeting pipeline

Per effect, per tick interval: **range filter** (the aura sensor shape from the broadphase) → **faction/target-flag filter** (`targetsEnemies` / `targetsAllies` / `targetsStructures` / `targetsSelf`) → **selector sort** (`nearest` default, `lowest_health` by percentage, `all`) → take the first `maxTargets`.

There is deliberately **no line-of-sight filter** — auras pass through walls. That was cut in 2026-07-10 with a written rationale (walls remain movement blockers; wall-cheese is handled by mob AI steering and leashing instead). Don't re-add it without reading `tdt.md §4.2`.

"No friendly fire" is not special-cased anywhere; it is simply `targetsAllies: false`. Heal auras never heal the caster.

4.4 Two multiplicative scaling axes

This trips people up, so it is worth stating precisely:

- **Specialisation axis** — per-skill level: `base + (level-1) × perLevel (skills.Scaled)`. Authored per skill.
- **Inflation axis** — per character level: `f(L) = growth^(L-1)`, one shared formula in `pkg/aura/curve`, config-driven (`levelGrowth 1.12 × maxLevel 30 ≈ 27× total`).

The inflation axis passes through a **single seam** in the `SkillSystem` (`casterPowerScale`) and multiplies **HP-side output only** — damage, heal, dot, hot, shield absorb, self-heal, self-damage cost. It never touches radius, tick rate, target count, or the relative vocabulary (crit, execute, berserker, lifesteal, slow, resist).

Mobs carry no runtime level multiplier. Mob JSON authors `tier` (normal/elite/boss — a pure label for framing/VFX) + `curveLevel` (position on f) + `factors.baseMaxHealth`; the loader derives max HP and the mob's power scale. Writing a raw `factors.maxHealth` **hard-fails at load** — deliberately, so a growth-curve change stays a one-knob re-derivation.

4.5 Unlocks and combinations

Five data-driven ways to obtain a skill: **milestone** (level gates, `api/milestones/`), **mob kill drops** (`unlocks` array on the mob JSON, with a chance), **NPC teaching** (walk into an NPC's sensor), **world exploration** anchors, and **combination recipes** (`api/recipes/`, 10 curated). Recipes are **curated, not algorithmic, and not documented in-game** — the community is meant to discover them. A recipe result can itself be an ingredient for a higher recipe.

4.6 Damage tags and resistances

Damage effects carry **arbitrary string tags** (`fire`, `physical`, or something bespoke like `boss_x_lava`), and resist sources list the tags they cover. Deliberately not a fixed enum, so a one-off hazard needs content, not a code change. Multipliers compose (`skills.ResistMultiplier`).

5. Content pipeline

All tunable content is JSON under `api/`, loaded by registries that mirror each other in shape. As of the milestone-table move, `-content ../api` covers **100%** of tunable content — no rebuild needed for a balance change.

DIRECTORY	COUNT	NOTES
<code>api/skills/</code>	47 + 36 mob	Registry pin test asserts the total; bump it when you add one.
<code>api/mobs/</code>	50	tier + curveLevel + baseline stats + skills + unlock drops.
<code>api/factions/</code>	12	Hostility relations, <code>friendlyToPlayers</code> .
<code>api/recipes/</code>	10	Combination unlocks.
<code>api/props/</code> , <code>api/zones/</code>	5 / 2	World geometry. <code>world.json</code> holds ~777 props, ~471 spawns, campfires, NPCs — authored in the in-game zone editor, not by hand.
<code>api/milestones/</code>	1	Level → guaranteed unlock table.

THE VALIDATION PHILOSOPHY IS THE THING TO PRESERVE

`skills/definition.go` hard-fails at boot on anything that would be a silent no-op: variance on a non-rolling effect, resist fields on a non-resist effect, empty or duplicate tags. On top of that, `validateEffectKeys()` is a **per-effect-type key allowlist with rename hints** — an unknown or stale field in a skill JSON fails the boot naming both the bad field and its replacement. That one function kills the most common authoring bug class, and is why the JSON pipeline is trustworthy without a schema. Extend it when you add an effect type.

6. Wire protocol

WebSocket + **FlatBuffers** (flatc v24.3.25), namespace `AuraApi`. Schemas in `api/schema/`:

- `client.fbs` — `Input`, `Join`, `Cheat`, `ChatMessage`
- `server.fbs` — `GameState`, `Welcome`, `Obituary`, the `EntityType` enum, entity tables
- `common.fbs` — `Vec2f`, shared enums

After editing a `.fbs`, regenerate **both** sides: `cd api/schema && ./make.sh`, then `make -C backend gen`. Schema changes should be **append-only** where possible — several shipped features (aura category rings, tier frames, applied-effect pips) added exactly one field at the end of an existing table and cost nothing on the client.

Note the `-2` sentinel on `active_aura_slot`: FlatBuffers omits default values, so an explicit `-1` is indistinguishable from an absent field. `-2` means "deactivate". It stays; the paired constants are `model.ActiveAuraSlotDeactivate` and `DEACTIVATE_AURA_SLOT`.

7. Frontend

TypeScript + webpack + PixiJS, organised as feature modules under `frontend/src/features/`: `backend/` (websocket, deserialisation, entity snapshots), `core/` (game loop, entity manager), `game-objects/` (Pixi render classes for mobs, characters, corpses, placeables), `camera/`, `controls/`, `input-system/`, `darkness/`, `day-cycle/`, `mini-map/`, `user-interface/`, `zone-editor/`, `internal-tools/` (dev panel, only with `?develop`).

The client is a **pure renderer of server state** plus input capture. Nothing combat-relevant is computed client-side; the server is authoritative for everything that matters.

7.1 Two rules that are not obvious from the source

- **Use `pointerdown`, never `click`, on HUD panels.** `MouseManager` registers a `mousedown` listener on `document.documentElement` with `preventDefault()`, which suppresses the synthetic `click`. A `click` handler on a HUD panel silently never fires.
- **Fixed field of view.** How much world you see is a game constant, not a browser artefact — zoom levels are defined as visible world *heights* and the camera scales to cover. Browser zoom only changes sharpness. This closes the fairness hole where zooming out granted sight range, and it is capped below the server's 20×12 streaming viewport so you can never outrun the entity stream.

7.2 The house answer to frontend/backend duplication

The client used to mirror the backend skill registry in a hand-synced `skills.ts` table. That is gone. The server now exposes `GET /skills` and `GET /mobs` serving its **parsed** registries, and the client fetches them. **Serve a catalog, don't mirror a table** — apply this to any new content duplication you are tempted to introduce.

8. What the player actually sees

How the systems above surface in the client, roughly in the order a new player meets them:

SYSTEM	PLAYER-FACING FORM
Active aura	A circle drawn around your character at its real radius, with a coloured ring band per <i>effect category</i> (damage, heal, shield, slow, light, resist). Ticks are visible as hit VFX on the struck target — a slash for slow ticks, fire for fast ones — so the circle reads as <i>range</i> , not as a hit zone.
Mobs	Same aura rings (you can read a mob's threat from its ring colour and size before it reaches you). Tier is a frame around the mob: silver for elite, gold for boss. Nameplates show <code>Name cL</code> tinted WoW-style by level difference — red $\geq +5$ down to gray ≤ -6 .
Resource	One bar. Overhead bar on every entity, plus the HUD bar for yourself. Buffs/debuffs currently applied to an entity show as coloured pips under its overhead bar.
Cooldowns	Ability bar with hotkeys, a cast/cooldown bar stack above it, and a burst VFX on fire.
Spellbook	Panel listing everything discovered, with hover tooltips showing current → next values (fed from <code>GET /skills</code>). Click a skill then press a slot hotkey to bind it; Escape cancels.
Unlocks	Glow on the spellbook + an alert banner. OPEN Source attribution ("taught by X") is not implemented — testers reported never noticing where abilities came from. This is the next queued work item.
Darkness & light	Authored dark areas (caves, the zone-1→2 tunnel) render as an opaque overlay with holes punched by light sources: your own small self-glow, a Lantern aura, campfires. In unlit darkness, mob nameplates are hidden. The tunnel is the designed tutorial for role support — light costs you your damage aura, so someone else has to carry it.
Campfires	Hard safe zones, heal over time, and act as respawn anchors.
NPCs	Peaceful; walking into their 1.0-unit sensor triggers a chat bubble (10 s) and can teach a skill. Often placed next to harvest-mobs only one specific aura can damage — a soft profession identity without a class system.
Death	Corpse + respawn button. Passive regen is combat-gated. No XP loss; all combat participants get XP (no formal groups in v1).
Boss	The Ork Warlord: a scripted encounter with phases, a threat table, and adds, announced to the whole server via the alert banner.

9. Testing and verification

Test-to-production ratio is about **0.96:1** (~22k lines of test code, 81 test files). The discipline is real: regressions found in play get pinned with a named test. **TDD is the expected workflow** for game logic and bug fixes — write the failing test first.

LAYER	WHAT IT IS
<code>go test ./...</code>	Runs clean in seconds. Now gated in CI alongside <code>go vet ./...</code> and <code>npm run typecheck</code> .
<code>cmd/simharness</code>	A balance explorer, not a test suite: TTK/TTD batteries, 1-vs-N matrices, kills-per-hour chains per level bracket, plus a web UI. It also holds a deterministic guardrail test suite that asserts balance invariants (e.g. band ordering, the power ceiling) — new content is expected to pass it.
<code>cmd/loadbot</code>	Real websocket clients with real skills, for capacity measurement.
<code>*_alloc_test.go</code>	Allocation pins asserting zero allocations per tick in the hot broadphase paths. These exist because the idle loop once allocated 11 MB/s and overloaded an empty server.
Playwright smoke	Headless browser drives the real HUD end to end.

Before declaring anything done: `go build ./...`, the relevant `go test`, and — for anything with a runtime surface — an actual in-game check. Report the output rather than asserting success.

10. Deployment

One Hetzner box. A single `aurad` process serves the frontend bundle, the websocket, and the HTTP catalog endpoints, with autocert TLS and a port-80 redirect, under systemd with restart-on-failure. `devops/deploy.sh` does a full deploy or `--content-only`. Cheat tokens are server-side only (`/opt/aurad/tokens.list`).

THERE IS NO PERSISTENCE

Every restart wipes all characters. Announce before deploying. This is the single biggest open gap and the next major work item — accounts and persistence, with character sacrifice / meta-progression as its first consumer.

11. Known challenges

11.1 Structural, on the roadmap

CHALLENGE	STATE
No persistence, no accounts	Nothing survives a restart. Direction decided (anonymous-first, upgrade path); database choice, migration framework, and snapshot strategy all open. This is next .
Single-threaded ceiling	Fine for realistic v1 load. The escape hatch (one Space per zone) is designed but unbuilt. Note that a Space boundary is a hard wall in the simulation — no aura crosses it — so seams have to sit at chokepoints where nothing is meant to fight across. That is exactly why the zone topology is tunnels.
Clustered-density wall	~60-70 players on one screen. Broadphase is $O(n^2)$ per cell and <code>bruteIntersectShapes</code> is ~39% of loop CPU; the send path is the other half. Not urgent — the game's shape (no PvP, ~5-20 players at a boss) does not produce big clusters naturally.
Zone chat	Chat exists; zone scoping does not.
Combat-feel SFX	Deferred — the audio scaffold exists, no placeholder assets available.

11.2 Live-ops watch items

- **Tick overload on the live host.** The production vCPU is ~3.4× slower than the dev box on identical idle work, which lands the tick tail right on the 33 ms line. An allocation fix took idle CPU from 25.8% to 17% of a core, but the verdict needs a day of live logs to confirm. Nothing here reproduced locally, even with `GOMAXPROCS=2` — it is raw per-core speed, not core count.
- **Day/night cycle is switched off.** `DAY_CYCLE_PRESENTATION_ENABLED = false`. It made avatars invisible at the day→night transition; never reproduced headlessly, looks GPU-dependent. Prime suspect is ~25 per-layer `ColorMatrixFilter` passes. Don't re-enable without collapsing those into one pass on a single parent container.
- **Unlock source attribution.** Players do not learn where a new ability came from. Proposed fix: a server-authored per-unlock alert from all five grant sites over the existing announcement channel, no schema change.

11.3 Code-level friction you will hit

- `sys/skills.go` is ~1,600 lines / ~40 functions — a flat dispatch that grows with every effect type. Readable today. The written guidance is *not* to refactor it pre-emptively, but it is where the next structural pressure will show.
- The `addXxx` registration methods in `core/game.go` — eight near-identical type-switch loops. The most error-prone backend extension point: adding an entity type means touching all the relevant ones.
- Three logging styles coexist — `slog`, `log.Printf`, `fmt.Printf`. Prefer `slog`.
- `go:embed` patterns do not recurse. A pattern needs both `*.json` and `**/*.json`. Disk-based registry tests cannot catch embed gaps, so there is an explicit embed pin test. Before any manual test: kill the old process (`kill -x aurad`), rebuild, and check the boot log counts — a stale process silently masks new behaviour.
- Adding an entity type still touches three places: mob JSON, the `.fbs` enum, and the frontend render-class map. The map is now keyed by enum member (so TypeScript enforces exhaustiveness), but the three-place sync itself remains.
- Frontend has no linter and ~18 stray `: any`. Typecheck runs in CI; ESLint does not exist.
- WARP has 1-unit granularity — it integer-divides before the float cast. Since NPC sensors dropped to 1.0 it can no longer reliably land you inside an NPC trigger; warp to the nearest whole unit and walk the last step.

12. How to work in this repo

The working style is documented in `CLAUDE.md` and is enforced socially rather than by tooling. The parts that will affect you immediately:

- **Plan before code.** Non-trivial work is designed plan-first in a `docs/plan-*.md` doc — what changes and why, chunk breakdown, decisions, open questions, test strategy — and then executed one chunk at a time, referencing the plan.
- **Every number is a placeholder until explicitly marked final**, and even "FINAL" means "first-pass settled", not frozen. Damage values, radii, drop rates, level caps, respec costs — all of them. Don't treat a value in the code as a decision that was made.
- **KISS / DRY / YAGNI, seriously applied.** Build what the current design requires, not what future combinations might. This is stated explicitly for the aura system.
- **Extend, don't rewrite** the inherited physics, collision, and WebSocket/FlatBuffers layers — they are treated as stable foundations.
- **Status lives in `CLAUDE.md`, not in the design docs.** The GDD and TDD are the big picture; on detail conflicts, `CLAUDE.md`, the roadmap, and the plan docs win.

WHERE TO START READING

`CLAUDE.md` (current state, open items) → `docs/gdd.md` §4–5 (combat pacing, power curve) → `docs/tdd.md` §4 (system architecture and the decision records) → `docs/architecture.md` (cost model, zones as Spaces) → then `backend/pkg/aura/sys/skills.go` alongside a couple of skill JSONs. For content work, `docs/manual-content-authoring.md` is the authoritative procedure.